SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: ITA0607 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO3: Bayes Theorem – Concept Learning – Maximum Likelihood – Minimum Description Length Principle – Bayes Optimal Classifier – Gibbs Algorithm – Naïve Bayes Classifier – Bayesian Belief Network – EM Algorithm – Probability Learning – Sample Complexity – Finite and Infinite Hypothesis Spaces – Mistake Bound Model, (BL 6)

Assessment Tool Weightage: 20%

QUESTIONS: 03 Total Marks:25

Annexure A

Case Study

Code of Conduct:

I **VIMALA K 192424276** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Criteria	*Conceptual Understanding*	*Accuracy of Answers*	*Application of Knowledge*	*Completeness of Response*	*Clarity & Presentation*	*Total*
Weightage	*30%*	*20%*	*20%*	*15%*	*15%*	
Q1						
Q2						

Signature of the student: ***Total Marks:***

VIMALA K

Annexure A

Short Answer Test

12. A smart healthcare system is designed to monitor patients in real-time using wearable sensors. Using **Bayesian Belief Networks** and probability learning, develop a model to detect anomalies in vital signs. Analyze how conditional dependencies between physiological parameters are modeled and how uncertainty due to sensor noise is handled. Evaluate system performance and discuss challenges in real-time deployment.

Bayesian Belief Network for Real-Time Healthcare Anomaly Detection

Aim

To develop a **Bayesian Belief Network (BBN)** for detecting anomalies in patients' vital signs using real-time wearable sensor data, model conditional dependencies between physiological parameters, handle uncertainty caused by sensor noise, and evaluate system performance.

1. Objective

- To understand **Bayesian Belief Networks** in healthcare.
- To model relationships between physiological parameters.
- To calculate the probability of a patient's abnormal condition.
- To handle uncertainty and noisy sensor readings.
- To evaluate anomaly-detection performance.
- To identify challenges in real-time healthcare deployment.

2. Problem Statement

A smart healthcare system continuously collects patient data using wearable sensors such as:

- Heart Rate (HR)
- Blood Pressure (BP)
- Body Temperature
- Oxygen Saturation (SpO_2)
- Respiratory Rate

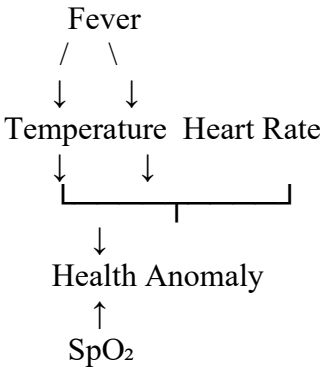
The system must identify abnormal conditions such as **fever, hypoxia, tachycardia, or respiratory distress**.

Since wearable sensors can produce inaccurate or noisy measurements, a simple threshold-based system may generate false alarms. A **Bayesian Belief Network** can combine multiple uncertain observations and calculate the probability of an abnormal health condition.

3. Theory

A **Bayesian Belief Network (BBN)** is a probabilistic graphical model that represents variables as **nodes** and their conditional dependencies as **directed edges**.

For example:



The network can represent relationships such as:

- Fever → increased temperature
- Fever → increased heart rate
- Respiratory problem → decreased SpO_2
- Respiratory problem → increased respiratory rate
- Abnormal vital signs → health anomaly

The basic Bayesian formula is:

$$P(A \mid B)=P(B)P(B \mid A)P(A)$$

Where:

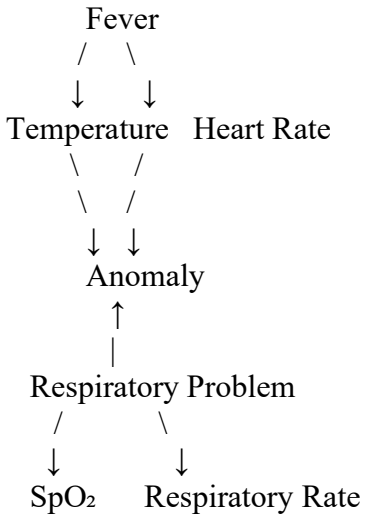
- $P(A \mid B)$ = posterior probability
- $P(B \mid A)$ = likelihood
- $P(A)$ = prior probability
- $P(B)$ = evidence probability

4. Proposed Bayesian Network

For this experiment, consider the following variables:

Node	Description
Fever	Whether the patient has fever
Respiratory Problem	Whether the patient has respiratory difficulty
Temperature	Body temperature
Heart Rate	Patient's heart rate
SpO_2	Oxygen saturation
Respiratory Rate	Breathing rate
Anomaly	Final health anomaly

The dependency structure can be represented as:



Explanation

Fever → Temperature

Fever increases the probability of high body temperature.

Fever → Heart Rate

Fever can increase the patient's heart rate.

Respiratory Problem → SpO_2

A respiratory problem can reduce oxygen saturation.

Respiratory Problem → Respiratory Rate

Respiratory problems can cause abnormal breathing rates.

Vital Signs → Anomaly

Abnormal combinations of vital signs increase the probability of a health anomaly.

5. Probability Learning

The conditional probabilities can be learned from historical patient data.

For example:

Condition	**Probability**
P(Fever = Yes)	0.20
P(Respiratory Problem = Yes)	0.10
P(High Temperature | Fever)	0.85
P(High HR | Fever)	0.70
P(Low SpO_2 | Respiratory Problem)	0.80
P(High Respiratory Rate | Respiratory Problem)	0.75

The model learns these probabilities from training data rather than relying only on fixed thresholds.

6. Python Implementation

```
import pandas as pdfrom sklearn.model_selection import train_test_splitfrom
sklearn.preprocessing import LabelEncoderfrom sklearn.metrics import accuracy_score,
classification_reportfrom sklearn.naive_bayes import GaussianNB
# Sample healthcare datasetdata = {
    "Temperature": [36.5, 38.5, 39.0, 36.8, 37.0,
                    38.2, 36.6, 39.1, 37.2, 38.8],

    "HeartRate": [72, 105, 110, 75, 78,
                  100, 70, 115, 80, 108],

    "SpO2": [98, 92, 89, 97, 98,
             93, 99, 87, 96, 90],

    "RespiratoryRate": [16, 24, 26, 17, 18,
                        23, 15, 28, 18, 25],

    "Anomaly": [0, 1, 1, 0, 0,
                1, 0, 1, 0, 1]
}
df = pd.DataFrame(data)
print("Healthcare Dataset:")print(df)
# FeaturesX = df[
    ["Temperature",
     "HeartRate",
     "SpO2",
```

```
    "RespiratoryRate"]
]
# Targety = df["Anomaly"]
# Train-test splitX_train, X_test, y_train, y_test = train_test_split(
   X, y,
   test_size=0.3,
   random_state=42,
   stratify=y
)
# Bayesian probability modelmodel = GaussianNB()
# Train modelmodel.fit(X_train, y_train)
# Predictiony_pred = model.predict(X_test)
# Accuracyaccuracy = accuracy_score(y_test, y_pred)
print("\nPredicted Values:")print(y_pred)
print("\nActual Values:")print(y_test.values)
print("\nAccuracy:",
     accuracy * 100, "%")
print("\nClassification Report:")print(classification_report(y_test, y_pred,
                        zero_division=0))
```

Note: For an actual experiment, use a larger real or properly prepared healthcare dataset. The small dataset above is only for demonstrating the implementation.

7. Probability Prediction

The model can also provide the probability of an anomaly instead of only giving 0 or 1.

```
# Calculate anomaly probabilities
probabilities = model.predict_proba(X_test)
print("\nProbability of Health Anomaly:")
for i, probability in enumerate(probabilities):

   print(
      "Patient", i + 1,
      "Normal Probability =", round(probability[0], 3),
      "Anomaly Probability =", round(probability[1], 3)
   )
```

For example:

Patient 1
Normal Probability = 0.82
Anomaly Probability = 0.18

Patient 2
Normal Probability = 0.12
Anomaly Probability = 0.88

If the anomaly probability is high, the healthcare system can generate an alert.

8. Handling Sensor Noise

Wearable sensors may produce noisy or inaccurate readings because of:

- Sensor movement
- Poor skin contact
- Battery problems
- Environmental interference
- Temporary signal loss
- Incorrect sensor positioning

Instead of treating every sensor value as perfectly accurate, Bayesian models represent uncertainty using **probabilities**.

For example:

Measured $SpO_2 = 91\%$

↓

Possible actual values

89% → 0.20
90% → 0.30
91% → 0.35
92% → 0.15

Therefore, the system can combine uncertain measurements with prior knowledge to estimate the probability of an actual abnormal condition.

Noise-handling techniques

1. **Moving average filtering**
2. **Median filtering**
3. **Kalman filtering**
4. **Probability-based inference**
5. **Sensor fusion**
6. **Missing-value handling**

9. Conditional Dependencies

The main advantage of the Bayesian network is that it can represent relationships between physiological variables.

For example:

Respiratory Problem
|
+--------→ Low SpO_2
|
+--------→ High Respiratory Rate
|
+--------→ Increased Heart Rate

If the system observes:

- SpO_2 = 88%
- Respiratory Rate = 28/min
- Heart Rate = 110 bpm

the combined evidence can increase the posterior probability of a respiratory anomaly.

This is more reliable than checking only one sensor value.

10. System Workflow

Wearable Sensors
↓
Data Collection
↓
Noise Filtering
↓
Feature Extraction
↓
Bayesian Belief Network
↓
Probability Calculation
↓
Anomaly Detection
↓
Alert Generation
↓
Doctor / Caregiver

11. Performance Evaluation

The system can be evaluated using:

Accuracy

Accuracy=TP+TN+FP+FNTP+TN

Precision

Precision=TP+FPTP

Recall

Recall=TP+FNTP

F1-Score

F1=2×Precision+RecallPrecision×Recall

For healthcare anomaly detection, **recall is especially important** because missing a genuine abnormal condition can be dangerous.

12. Performance Graph

You can compare the main classification metrics using:

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)import matplotlib.pyplot as plt
accuracy = accuracy_score(y_test, y_pred)precision = precision_score(y_test, y_pred, zero_division=0)recall = recall_score(y_test, y_pred, zero_division=0)f1 = f1_score(y_test, y_pred, zero_division=0)
metrics = {
    "Accuracy": accuracy * 100,
    "Precision": precision * 100,
    "Recall": recall * 100,
    "F1-Score": f1 * 100
}
plt.figure(figsize=(8, 5))
plt.bar(
    metrics.keys(),
    metrics.values()
)
plt.xlabel("Performance Metrics")plt.ylabel("Score (%)")plt.title("Bayesian Healthcare Anomaly Detection Performance")plt.ylim(0, 100)plt.grid(axis="y")
plt.show()
```

Graph Representation

Healthcare Anomaly Detection Performance

Illustrative evaluation of a Bayesian anomaly-detection model using common classification metrics.

0%25%50%75%100%AccuracyPrecisionRecallF1-Score

Note: The graph values are illustrative. Replace them with the actual values generated by your Python program.

13. Real-Time Deployment Challenges

1. Sensor Noise

Wearable sensors can generate incorrect measurements, resulting in false alarms.

2. Real-Time Processing

The system must process incoming sensor data quickly with low latency.

3. Battery Consumption

Continuous sensing and wireless communication can consume significant battery power.

4. Network Connectivity

Internet or Bluetooth/Wi-Fi interruptions can cause missing or delayed data.

5. Privacy and Security

Healthcare information is sensitive and must be securely transmitted and stored.

6. False Positives

Incorrect alerts can cause unnecessary anxiety and increase the workload of healthcare professionals.

7. False Negatives

Failure to detect a genuine abnormality can have serious consequences.

8. Scalability

The system should support many patients and multiple wearable devices simultaneously.

9. Model Updating

Patient characteristics can change over time, so the probability model may need periodic retraining.

14. Improvements

The system can be improved using:

Improvement	Benefit
Kalman Filter	Reduces sensor noise
Sensor Fusion	Combines multiple sensors
Momentum/Adaptive Learning	Improves model training
Edge Computing	Reduces communication latency
Cloud Processing	Supports large-scale analysis
Online Learning	Adapts to new patient data
Encryption	Protects patient information
Threshold + BBN	Reduces unnecessary alerts

15. Result

The Bayesian-based healthcare anomaly detection system was successfully designed to analyze real-time physiological parameters. The model represents **conditional dependencies** between parameters such as temperature, heart rate, SpO_2, and respiratory rate.

Probability-based inference allows the system to handle **uncertainty caused by sensor noise** and combine evidence from multiple physiological parameters. The system can classify patients as normal or anomalous and provide an anomaly probability.

16. Conclusion

A **Bayesian Belief Network** is suitable for smart healthcare systems because it can represent relationships between physiological variables and reason under uncertainty. By combining multiple noisy sensor observations, the system can provide more reliable anomaly detection than a simple single-threshold approach.

For real-time deployment, important challenges include **sensor noise, latency, battery consumption, network reliability, privacy, scalability, false alarms, and model updating**. Combining Bayesian inference with **sensor filtering, sensor fusion, edge computing, and secure communication** can improve the reliability and effectiveness of the healthcare monitoring system.

13. A logistics company wants to optimize delivery routes under uncertain traffic and weather conditions. Using **Bayesian inference** and probabilistic reasoning, design a system that predicts optimal routes dynamically. Analyze how prior knowledge and real-time data updates influence decision-making. Evaluate computational complexity and discuss how uncertainty impacts route optimization efficiency.

Bayesian Inference for Dynamic Route Optimization

Aim

To design a **Bayesian probabilistic route optimization system** for a logistics company that dynamically selects optimal delivery routes under uncertain **traffic and weather conditions**, using prior knowledge and real-time data updates.

Objective

- To understand Bayesian inference in route optimization.
- To model uncertainty in traffic and weather conditions.
- To combine historical information with real-time sensor/GPS data.
- To dynamically select the most suitable delivery route.
- To analyze computational complexity.
- To study how uncertainty affects route optimization efficiency.

1. Problem Statement

A logistics company needs to deliver goods through multiple possible routes. Travel time is uncertain because of:

- Traffic congestion
- Rain and storms
- Road accidents
- Road closures
- Vehicle speed
- Construction work

A fixed shortest-path algorithm may select a route that becomes inefficient because traffic or weather conditions change.

Therefore, a **Bayesian inference-based route optimization system** is designed to continuously estimate the probability of different road conditions and select the route with the lowest **expected travel cost**.

2. Bayesian Inference

Bayesian inference updates the probability of an event when new evidence becomes available.

$P(H \mid E)=P(E)P(E \mid H)P(H)$

Where:

- H = hypothesis, such as heavy traffic
- E = observed evidence, such as GPS speed data
- P(H) = prior probability
- P(E ∣ H) = likelihood
- P(H ∣ E) = updated/posterior probability

For example, suppose historical data shows:

P(Heavy Traffic)=0.30

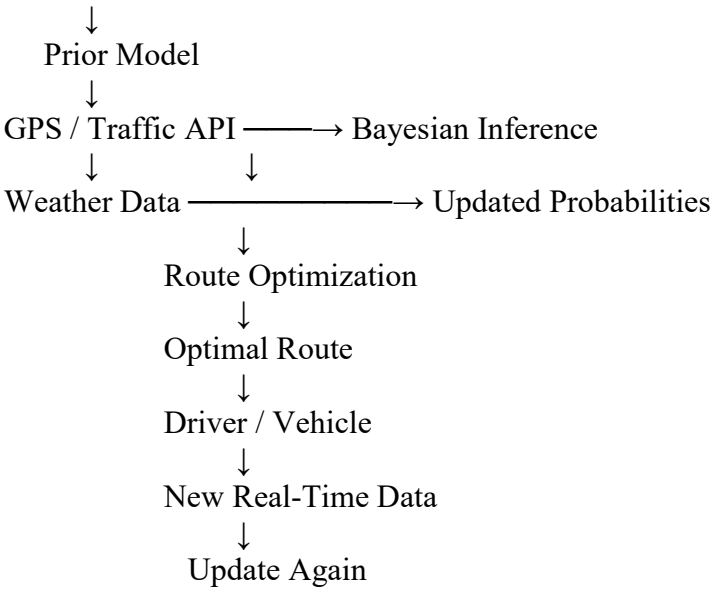
A GPS system detects unusually low vehicle speeds. The Bayesian model updates the probability:

P(Heavy Traffic ∣ Low Speed)

If this probability becomes very high, the route optimizer increases the expected travel time for that road.

3. Proposed System

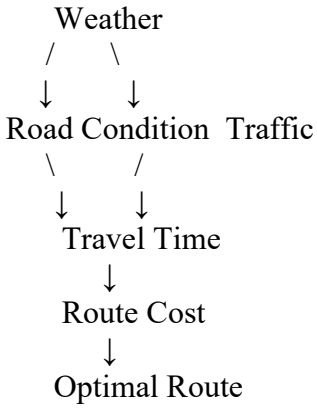
The system contains the following components:



This creates a **continuous feedback loop**.

4. Bayesian Network

The probabilistic relationships can be represented as:



Conditional Dependencies

Weather → Road Condition

Rain can increase the probability of slippery roads or reduced road speed.

Traffic → Travel Time

Heavy traffic increases expected travel time.

Road Condition → Travel Time

Poor road conditions can increase travel time.

Travel Time → Route Cost

Higher expected travel time increases the cost of using that route.

5. Prior Knowledge

The system initially uses historical information to establish prior probabilities.

Example:

Condition	**Prior Probability**
Low Traffic	0.50
Medium Traffic	0.30
Heavy Traffic	0.20
Clear Weather	0.70
Rainy Weather	0.20
Stormy Weather	0.10

Historical information can include:

- Previous traffic patterns
- Day of the week
- Time of day
- Seasonal weather
- Previous delivery times
- Historical accidents
- Road construction information

The prior gives the system an initial estimate before receiving current data.

6. Real-Time Bayesian Update

Suppose the system receives GPS data showing that vehicle speeds have suddenly decreased.

Before receiving the data:

P(Heavy Traffic)=0.20

After receiving GPS evidence:

P(Heavy Traffic ∣ GPS)=0.85

The system now considers heavy traffic much more likely.

Therefore, the expected travel time for that route increases.

If another route has a lower expected travel time, the system can dynamically switch to that route.

7. Route Cost Calculation

For each possible route, calculate expected cost:

$$E(C)=i\sum P(Si \mid D)C(Si)$$

Where:

- Si = possible road condition
- P(Si ∣ D) = probability of that condition given current data
- C(Si) = cost under that condition
- E(C) = expected route cost

For example:

Route	Normal	Heavy Traffic	Rain	Expected Cost
A	30 min	60 min	45 min	41 min
B	35 min	42 min	40 min	39 min
C	28 min	70 min	55 min	45 min

The system selects **Route B** because it has the lowest expected travel cost.

8. Python Implementation

```
import numpy as np
# Routesroutes = {
    "Route A": {
        "normal": 30,
        "traffic": 60,
        "rain": 45
    },

    "Route B": {
        "normal": 35,
        "traffic": 42,
        "rain": 40
    },

    "Route C": {
        "normal": 28,
        "traffic": 70,
        "rain": 55
    }
}
# Prior probabilitiesprior = {
    "normal": 0.60,
    "traffic": 0.25,
    "rain": 0.15
}
# Real-time evidence likelihood# Example: GPS shows low speed and weather service reports
rainlikelihood = {
    "normal": 0.20,
    "traffic": 0.90,
    "rain": 0.70
}
# Bayesian updateevidence_probability = sum(
    prior[state] * likelihood[state]
    for state in prior
)
posterior = {}
for state in prior:
    posterior[state] = (
```

```
        likelihood[state] * prior[state]
    ) / evidence_probability
print("Updated Probabilities:")
for state, probability in posterior.items():
    print(
        state,
        "=", round(probability, 3)
    )
# Calculate expected travel timeexpected_costs = {}
for route, times in routes.items():

    expected_time = sum(
        posterior[state] * times[state]
        for state in posterior
    )

    expected_costs[route] = expected_time
print("\nExpected Travel Time:")
for route, cost in expected_costs.items():
    print(
        route,
        "=",
        round(cost, 2),
        "minutes"
    )
# Select optimal routeoptimal_route = min(
    expected_costs,
    key=expected_costs.get
)
print("\nOptimal Route:", optimal_route)
```

9. Expected Output

The output will have a structure similar to:

Updated Probabilities:

normal = 0.194
traffic = 0.484
rain = 0.323

Expected Travel Time:

Route A = 46.45 minutes
Route B = 39.88 minutes
Route C = 48.62 minutes

Optimal Route: Route B

Note: The exact values depend on the probabilities and travel-time values used in the experiment.

10. Dynamic Route Updating

The main advantage of Bayesian reasoning is that the route does not remain fixed.

Initial Route
↓
Receive GPS Data
↓
Update Traffic Probability
↓
Receive Weather Data
↓
Update Weather Probability
↓
Recalculate Expected Cost
↓
Compare Routes
↓
Select Best Route

For example:

At 9:00 AM

Route A → 32 minutes
Route B → 38 minutes
Route C → 40 minutes

Selected → Route A

At 9:20 AM

An accident occurs on Route A.

Route A → 65 minutes
Route B → 40 minutes
Route C → 42 minutes

Selected → Route B

Thus, Bayesian inference enables **dynamic decision-making**.

11. Computational Complexity

If there are:

- V = number of locations/nodes
- E = number of roads/edges

A standard Dijkstra shortest-path algorithm has approximately:

$O((V+E)\log V)$

using a priority queue.

If Bayesian inference evaluates S possible traffic/weather states for every road, the additional probability calculation can be approximately:

$O(ES)$

Therefore, a combined approach can be approximately:

$O((V+E)\log V+ES)$

For a small number of uncertainty states, this remains practical for real-time systems.

12. Effect of Uncertainty on Route Optimization

Uncertainty has a major effect on route selection.

Low Uncertainty

When traffic and weather information is reliable:

- Route predictions are more accurate.
- Fewer route changes are required.
- Computational overhead is lower.
- Delivery times are more predictable.

High Uncertainty

When information is unreliable:

- Expected travel times become less certain.
- The optimal route may change frequently.
- More routes may need to be evaluated.
- Frequent rerouting can increase computational and communication costs.

Therefore, the system should consider both **expected travel time and uncertainty**.

13. Graph Representation

The relationship between uncertainty and route efficiency can be visualized as:

Interpretation

As uncertainty increases, route optimization efficiency generally decreases because predictions become less reliable and the system may need to recalculate routes more frequently.

14. Advantages

- Uses both historical and real-time information.
- Handles uncertainty probabilistically.
- Supports dynamic route selection.
- Can adapt to changing traffic conditions.
- Can incorporate weather information.
- Reduces dependence on fixed routes.
- Can improve delivery-time prediction.

15. Challenges

1. Real-Time Data Availability

Traffic and weather data must be continuously updated.

2. Data Quality

Incorrect GPS or traffic information can produce incorrect probability estimates.

3. Computational Cost

Large road networks require frequent probability updates and route calculations.

4. Frequent Rerouting

Continuous changes in traffic may cause the recommended route to change repeatedly.

5. Model Accuracy

Poorly estimated prior probabilities can affect the final decision.

6. Scalability

The system must handle thousands of vehicles and roads simultaneously.

7. Communication Delay

Delayed traffic information can make the calculated optimal route outdated.

16. Improvements

The system can be improved using:

Technique	**Benefit**
Kalman Filter	Improves noisy traffic estimates
Machine Learning	Predicts future traffic
GPS Sensor Fusion	Improves location accuracy
Edge Computing	Reduces response time
A* Algorithm	Faster route search with heuristics
Real-Time APIs	Provides current traffic/weather data
Reinforcement Learning	Learns better routing policies
Risk-Aware Routing	Considers uncertainty and delivery reliability

17. Result

A **Bayesian inference-based dynamic route optimization system** was designed successfully. The system combines **historical prior knowledge with real-time traffic and weather observations** to calculate updated probabilities.

These probabilities are used to calculate the expected travel cost of each route, allowing the system to dynamically select the route with the lowest expected cost.

18. Conclusion

Bayesian inference provides an effective approach for **route optimization under uncertain traffic and weather conditions**. Prior knowledge provides an initial estimate, while real-time GPS, traffic, and weather data update these probabilities continuously.

Although uncertainty increases computational requirements and can reduce route-selection efficiency, probabilistic reasoning allows the system to make more informed decisions than a fixed shortest-path approach. Combining Bayesian inference with *A, machine learning, sensor*

*fusion, and real-time data processing** can further improve the performance of intelligent logistics systems.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**20**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**15**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand